

Privilege Escalation

Unit 15 — Module 4: Post-Exploitation (Linux & Windows basics)

Getting a shell is just the beginning. Most footholds are **low-privilege** — now we go from a normal user to **root**.

Where we are

- So far you've focused on **getting in** — recon, scanning, web bugs, exploits, passwords.
- But landing on a machine is usually just the **beginning**.
- Your first foothold is almost always a **low-privilege** account that can barely do anything.

This unit is about what happens *after* you get a shell: **post-exploitation**, and specifically **privilege escalation**.

Learning objectives

By the end of this unit you can:

- **Define** post-exploitation and privilege escalation; tell a low-priv user from root/admin.
- **Explain** why **enumeration** is the single most important privesc skill.
- **Run** core Linux enum commands (`id`, `whoami`, `sudo -l`, SUID `find`) and interpret them.
- **Identify & exploit** Linux vectors: **sudo misconfig**, **SUID**, **cron jobs**, **weak permissions**.
- **Describe** kernel exploits (awareness) and use **LinPEAS** responsibly.
- **Describe** Windows privesc basics: **unquoted service paths**, **WinPEAS** (concept).
- **Recommend** layered defenses: **least privilege**, **patching**, **hardening**.

Vocabulary (1 of 2)

Term	Meaning
Post-exploitation	Everything you do <i>after</i> getting access.
Privilege escalation	Going from a limited account to a more powerful one.
Low-privilege user	An account with limited rights.
Root	The all-powerful account on Linux/Unix.
Administrator / SYSTEM	The most powerful accounts on Windows.
Enumeration	Systematically gathering details to find weaknesses.
<code>sudo</code> / <code>sudo -l</code>	Run as another user / list what you may run as root.

Vocabulary (2 of 2)

Term	Meaning
SUID binary	A program that runs with its owner's privileges (often root).
Cron job	A task Linux runs automatically on a schedule.
World-writable	A file anyone can modify — dangerous if root uses it.
Kernel exploit	An attack on the OS core; powerful but can crash the box.
LinPEAS / WinPEAS	Scripts that scan a system and flag privesc paths.
Unquoted service path	A Windows misconfig that lets an attacker slip in a program.
Least privilege	Give every account only the access it actually needs.
Hardening / Patching	Tighten the config / install updates that fix holes.

Why this unit matters

- A foothold as a low-priv user is often nearly **harmless** on its own.
- Privesc turns that toehold into **total control** of the machine.
- The same enumeration attackers use is what **defenders** use to clean up.

The difference between a scare and a catastrophe is whether privesc succeeds.

An analogy: the office building

- A **foothold** is getting into the lobby with a visitor badge.
- **Privesc** is finding the unlocked door to the **master key cabinet**.
- **Enumeration** is patiently checking every door — not smashing a wall.

You rarely break in to the top. You **find** the door someone left open.



Read this before anything else

The ethics & legal line

Privesc is the moment it gets serious

- Privesc takes you from "I'm in" to "**I own everything.**"
- Full admin means access to *every* file, *every* user, *every* secret.
- On a real system that's the line between a minor incident and a **catastrophic breach**.

Running this against any machine you don't own or have **written permission** to test is a serious crime under the **CFAA** — even if you "only looked."

Where we run it

- **Only** authorized **TryHackMe rooms** or the **isolated class lab**.
- These boxes are **deliberately broken** — built to be conquered safely.
- **Never** point LinPEAS, `find`, or any technique at a machine outside scope.

The same enumeration that finds a sudo misconfig is exactly what a **defender** runs to *remove* it before an attacker arrives.

Discussion: the out-of-scope server

A tester is hired to test a company's **web server**. Mid-test they find that with one command they could escalate to root on a **different** server — one **not** in the agreed scope. They're confident it would work.

- What should they do?
- Why doesn't "but I could have" make it okay?
- Where is the **authorization boundary**?

Day 1

Post-exploitation & why enumeration is everything

Warm-up

You just got a shell on a machine as user `bob`. `bob` can't read the password file or install software.

Now what?

Foothold $\neq$ owning the box

Term	Meaning
Post-exploitation	Everything you do <i>after</i> getting access
Privilege escalation	low-priv user → more powerful account
Low-privilege user	limited rights, can't touch system settings
Root	the all-powerful account on Linux
Administrator / SYSTEM	the most powerful accounts on Windows

A foothold is a low-priv user. Privesc is a **second, separate fight**.

Enumeration is the secret weapon

- The win is almost never a fancy exploit.
- It's **finding the misconfiguration the owner left behind**.
- The three first questions after you land:

```
whoami      # who am I?  
id          # what groups / uid?  
sudo -l     # what can I run as another user (often root)?
```

Privesc is won by *looking carefully*, not by magic.

Reading `id` output

```
$ id  
uid=1000(bob) gid=1000(bob) groups=1000(bob),4(adm)
```

- `uid=1000` → a **normal user**, not root (`uid=0`).
- `groups=` lists extra powers — `adm` can often read logs.

Group membership is a quiet lead. Always read the whole line.

Reading `sudo -l` output

```
$ sudo -l
```

```
User bob may run the following commands:
```

```
(root) NOPASSWD: /usr/bin/find
```

- `bob` can run `find` **as root**, with **no password**.
- That single line is often the whole path to root.

`NOPASSWD` plus a flexible program is a gift to an attacker.

Guided practice — predict the output

As a class, run / predict the output of:

```
whoami  
id  
sudo -l
```

- What does each one reveal?
- Define the vocabulary as it comes up.

Your turn (journal)

Read the Linux enumeration cheat-sheet. In your journal, write:

- the **three "first questions"** you ask after landing on a machine,
- and the **command** that answers each.

Check your understanding

You land on a box as `bob`. In **one sentence**, why is careful enumeration more valuable than hunting for a fancy exploit?

Answer

- The owner almost always **left a misconfiguration behind**.
- Finding that misconfig beats hoping a rare exploit exists.
- Privesc is won by **looking carefully**, not by magic.

When you're stuck, you skipped an enumeration step — not a better exploit.

Day 1 exit ticket

In one sentence: why is **enumeration** more important than any single exploit for privilege escalation?

Day 2

Linux vectors 1 & 2 — sudo misconfigurations and SUID binaries

Warm-up

`sudo -l` shows:

`(root) NOPASSWD: /usr/bin/find`

Why might that be **dangerous**?

Vector 1 — sudo misconfigurations

`sudo -l` shows what you can run as another user. Look for `NOPASSWD`:

```
(root) NOPASSWD: /usr/bin/find
```

- If you can run even a "harmless" program as root, it may let you **break out** to a root shell.
- Look it up on **GTFOBins** — a site listing how common binaries can be abused.

Abusing a sudo binary (GTFOBins)

```
sudo find . -exec /bin/sh \; -quit      # GTFOBins pattern → root shell
id                                       # confirm uid=0
```

- `find`'s `-exec` runs a command — as root, because of the sudo grant.
- Other classics: `sudo vim -c ':/bin/sh'`, `awk`, `less`, `python`.

Don't memorize — **look it up** on GTFOBins. That's exactly how professionals work.

Why that one line gives root

- `sudo` runs `find` **as root**.
- `find`'s `-exec` can launch **any** program — here, a shell `/bin/sh`.
- That shell inherits root, because its parent (`find`) was root.

A "harmless" tool becomes a root shell the moment it can run *another* program.

Vector 2 — SUID binaries

A **SUID binary** runs with its *owner's* privileges (often root), no matter who launches it.

```
find / -perm -4000 -type f 2>/dev/null      # list SUID programs
```

- Check each on **GTFOBins** for a **SUID** method.
- A SUID shell needs `-p` to *keep* the privileges:

```
/path/bash -p  
id                      # euid=0
```

28

Guided practice — trace why it works

- Look up one example on **GTFOBins** as a class (e.g., `find` or `vim`).
- Trace **why** that command spawns a root shell.

Your turn (lab)

1. Read the **Safety & authorization reminder** aloud.
2. Start the TryHackMe **Linux PrivEsc** room; get the low-priv shell.
3. Run `sudo -l` and the SUID `find`.
4. Attempt the **sudo** and/or **SUID** escalation. Record every command and result.

Check your understanding

`find / -perm -4000` lists a SUID copy of `bash` owned by root.

Why is that dangerous, and what flag do you need?

Answer

- A SUID `bash` runs with its **owner's** rights — root.
- You need `-p` so the shell **keeps** those privileges:

```
/path/bash -p  
id          # euid=0
```

Without `-p`, bash drops back to your normal user. The flag is the whole trick.

Day 2 exit ticket

Explain in your own words what a **SUID binary** is and why it matters for privesc.

Day 3

Linux vectors 3 & 4 — cron jobs and weak permissions
(+ kernel awareness)

Warm-up

If root runs a script every minute and *you* can edit that script...
what happens?

Vector 3 — cron jobs

A **cron job** is a task Linux runs automatically on a schedule.

```
cat /etc/crontab  
ls -la /etc/cron.d/ 2>/dev/null
```

If root runs a script **you can write to**, you can make root run *your* code.

Abusing a writable cron script

```
ls -la /path/to/scheduled-script.sh      # can you write it?  
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >> /path/to/scheduled-script.sh  
# wait for the cron interval to pass...  
/tmp/rootbash -p  
id
```

- The planted command runs **as root** when cron fires.
- It makes a SUID copy of bash; `-p` keeps root privileges.

Remember to **wait for the interval** before the planted command runs.

Vector 4 — weak / world-writable files

- **World-writable** = a file anyone can modify — dangerous if root later uses it.
- **Readable sensitive files** = a backup, config, or key that leaks a password.

```
ls -la /path/to/file          # check the rwx permissions (Unit 04!)
```

Find files you shouldn't be able to **write**, or shouldn't be able to **read**. Either can be a path to root.

Reading the danger in `ls -la`

```
-rwxrwxrwx 1 root root 142 backup.sh
```

- Owner is **root**, but the last `rwx` = **everyone can write** it.
- If root runs this script, your edits run **as root**.

The final three letters (`rwx` for "other") are where world-writable hides.

Kernel exploits (awareness only)

A **kernel exploit** attacks the core of the OS itself.

- Can grant **instant root**...
- ...but can **crash** the whole box.
- Usually a **last resort** — and the reason **patching** matters.

Prefer a quiet misconfiguration over a risky kernel exploit. This is awareness-level — we are not doing kernel exploit dev.

Guided practice — read a crontab

- Inspect a sample crontab and file listing as a class.
- Identify the **world-writable script root runs**.
- State the **plan** to abuse it.

Your turn (lab)

1. Continue the room.
2. Find and exploit the **cron** and/or **weak-permissions** vector to reach root.
3. Capture proof (`id` showing `uid=0`, the root flag).

Check your understanding

Root runs a world-writable script every minute via cron. You append a command to it.

Why must you **wait**, and what runs your code?

Answer

- The script only runs **when cron next fires** (e.g., each minute).
- When it does, it runs **as root**, so your appended command runs as root.
- A common payload: make a **SUID copy of bash**, then run it `-p`.

Edit, then wait for the interval. Impatience is the #1 cron mistake.

Day 3 exit ticket

Name the **four Linux vectors** we've covered and **one tell-tale sign** of each.

Day 4

Enumeration helpers (LinPEAS) + Windows privesc
basics

Warm-up

We found these issues by hand. What if a script could flag them all in 30 seconds — does that make the human unnecessary?

LinPEAS — the enumeration helper

```
./linpeas.sh | tee linpeas-output.txt
```

- Auto-scans and **color-codes** likely privesc paths (red/yellow = hot).
- It **finds** candidates — it does **not** think for you.
- You still verify, decide, and understand the manual checks.

It's a flashlight, not a brain.

Don't become "just a tool runner"

- Some LinPEAS findings are **false positives** or dead ends.
- If the tool isn't available, or fails, **you** still need the skill.
- You must be able to **explain** the actual vulnerability.

Reproduce at least one LinPEAS finding **by hand**, so you understand what it checked.

Windows privesc (basics / awareness)

Same idea as Linux: normal user → **Administrator / SYSTEM**, driven by misconfigurations.

- **Unquoted service path** — a service path with spaces lets an attacker slip in their own program.
- **WinPEAS** — the Windows version of LinPEAS.

Deep Windows / Active Directory attacks are **out of scope** this unit. We're just seeing the concept.

Guided practice — match LinPEAS to your hand-work

- Run **LinPEAS** on the lab box.
- Read its highlighted output together.
- **Match** its findings to the vectors you exploited by hand on Days 2–3.

Your turn (lab)

1. Run LinPEAS in the lab.
2. Annotate **which findings** led (or could lead) to root.
3. Note **at least one** finding you also found **manually**.

Check your understanding

A classmate says, "I'll just run LinPEAS and paste whatever it flags."
Give **two** reasons that's not enough.

Answer (any two)

- LinPEAS only **finds** candidates; it doesn't **decide** what's real.
- Some findings are **false positives** or dead ends.
- If the tool is missing or fails, **you** still need the skill.
- You must be able to **explain** the actual vulnerability.

Verify by hand, confirm the path, and understand *why* it works.

Day 4 exit ticket

Why shouldn't a pentester rely on **LinPEAS/WinPEAS alone**? Give one reason.

Day 5

The defense + document the path

Warm-up

You're now the sysadmin. Pick **one** vector from this week and **stop it**.

Defenses: attack by attack

Vector	Defense	Why it works
sudo misconfig	least privilege	remove broad / NOPASSWD grants
SUID binary	hardening (<code>chmod u-s</code>)	strip SUID from binaries that don't need it
cron + weak perms	hardening	fix file perms; only root writes root's scripts
kernel exploit	patching	keep kernel & packages current

Every vector you exploit must be paired with the fix that removes it.

Defense in depth

- **Least privilege** — don't grant sudo/SUID/admin rights that aren't needed.
- **Hardening** — fix file permissions, remove unnecessary SUID bits, quote service paths.
- **Patching/updates** — close kernel and software exploits.

Three layers. If one is missed, the others still slow the attacker down.

Check your understanding

Match each to its best defense: (a) a `NOPASSWD` sudo grant, (b) a world-writable cron script, (c) an out-of-date kernel.

Answer

- (a) sudo grant → **least privilege** (remove the broad grant).
- (b) cron script → **hardening** (fix the file permissions).
- (c) old kernel → **patching** (keep the kernel current).

Every vector you exploit has a specific fix. Name it in your report.

Document it: the privesc path

Write it so another person could **reproduce or fix** it:

- the **low-priv start** (`whoami` / `id`),
- each **enumeration step** and what it revealed,
- the **vector** used + exact commands,
- **proof of root** (`id` → `uid=0`, the flag),
- the **LinPEAS finding** that matched your manual work,
- a **defense** for each vector used.

A path without a remediation is **incomplete**. Attacks are paired with defenses.

Path rubric (abbreviated)

Criteria	Exemplary (4)
Reproducible path	Another person could follow it start → root, no gaps
Manual enumeration	Clear manual checks shown and interpreted
Proof of root	<code>id</code> uid=0 + flag
Defense / remediation	Specific, correct fix per vector
Ethics & scope	Authorized-target-only stated; professional tone

Day 5 exit ticket

Submit the privesc-path journal, plus one sentence:

"The misconfig that surprised me most was ____ because ____."



Lab walk-through (Days 2-5)

Platform: TryHackMe Linux PrivEsc room, from the **AttackBox** or **Kali** (optional basic Windows room for awareness).

1. Get the low-priv shell; confirm with `whoami` / `id`.
2. Enumerate: `sudo -l`, SUID `find`, crontab.
3. Look up leads on **GTFOBins**; exploit **at least two** vectors to **root**.
4. Run **LinPEAS**; match a finding to your manual work.
5. Document the **full path** + a **defense** for each vector.

Read the safety reminder aloud first. Every command runs on the **target box** in the authorized lab only.

Recap

- A foothold is low-priv; **privesc is a separate fight.**
- **Enumeration** wins — not a magic exploit.
- Linux vectors: **sudo misconfig, SUID, cron, weak permissions** (+ kernel awareness).
- **GTFOBins** = look it up, don't memorize.
- **LinPEAS** finds; the **human** decides.
- Windows: same idea, awareness-level (unquoted service paths, WinPEAS).
- Defenses: **least privilege, hardening, patching.**

Stretch goals

- Find a **second, different** path to root on the same box.
- Reproduce a LinPEAS red/yellow finding **entirely by hand** and explain it.
- Complete the optional **Windows PrivEsc** room; compare Linux vs. Windows privesc.
- Research a real **CVE-based kernel privesc** story (root cause + patch, no attack steps).

Exit ticket & discussion

Exit ticket: "The misconfig that surprised me most was ___ because ___."

Discussion: Why is **enumeration**, not exploitation, the heart of privesc? And: a tester *could* escalate root on a server **outside the agreed scope** — what should they do, and why doesn't "but I could have" make it okay?